B-YOLO: A Bayesian Learning Framework for YOLOv11s

Course Project Report
AI61004 - Statistical Foundations for AI/ML

Indian Institute of Technology Kharagpur

Group-2

Debraj Das	21ME3AI31
Subhasini Gupta	21AE3AI03
Aditya Ashok	21NA3AI39
Sachin Yadav	21CE3AI15
Bade Naga Manikanta Praveen	21ME3AI32

Submitted: May 3, 2026

Github Repo:
https://github.com/hari-hashing/Bayesian_YOLOv11s_Evaluations

Abstract

State-of-the-art detection models such as YOLOv11s are deterministic systems whose confidence scores lack probabilistic calibration, and whose weights are vulnerable to *catastrophic forgetting* when fine-tuned on new tasks. We present **B-YOLO**, a framework that replaces the deterministic feed-forward layers in the YOLOv11s detection head with Bayesian Neural Network (BNN) counterparts trained via Mean-Field Variational Inference. By representing weights as probability distributions rather than fixed scalars, the model can (i) report per-prediction uncertainty alongside standard metrics and (ii) leverage a Bayesian regularization strategy analogous to Elastic Weight Consolidation (EWC) to sequentially learn multiple vision tasks without degrading prior capabilities. We evaluate B-YOLO on four tasks - object detection, instance segmentation, pose/keypoint detection, and image classification - using COCO 2017 and Food-101. Our results demonstrate that the Bayesian head introduces negligible accuracy overhead while yielding meaningful uncertainty estimates: a Dice score improvement of +0.002 for segmentation and a Top-5 accuracy improvement of +1.13% for classification.

Contents

Project Notebook 2

Group Member Roles and Contributions 3

1 Introduction 4

2 Problem Statement 4

3 Methodology 5

4 Architecture 6

5 Loss Functions and Training Setup 7

6 Datasets 8

7 Implementation Details 8

8 Evaluation Metrics 9

9 Results 9

10 Discussion 12

11 Conclusion 12

A Mathematical Notation Summary 13

Project Notebook

The complete implementation, training runs, and experiments for this project are available in the following Google Colab notebook:

B-YOLO: Bayesian Learning Framework for YOLOv11s

https://github.com/hari-hashing/Bayesian_YOLOv11s_Evaluations

The notebook is organised into the following cells/sections:

1. Environment setup and package installation (`ultralytics`, `bayesian-torch`, etc.)
2. Backbone extraction and freezing from YOLOv11s
3. `BayesianLinear` layer implementation and unit tests
4. MC-Dropout2d segmentation head
5. COCO 2017 and Food-101 DataLoader pipelines
6. Training loops with KL annealing for classification and segmentation
7. Standard YOLOv11s inference for detection and pose tasks
8. MC uncertainty evaluation ($T = 5$ and $T = 10$ passes)
9. Results tables, bar charts, and uncertainty heatmap visualisations

Group Member Roles and Contributions

Division of responsibilities among Group-2 members.

Name	Roll No.	Primary Responsibilities
Debraj Das	21ME3A131	Data collection and preprocessing; sourced and prepared the COCO 2017 and Food-101 datasets; built the augmentation and normalisation pipelines; wrote the DataLoader modules for all four tasks.
Subhasini Gupta	21AE3A103	Bayesian architecture and training pipeline; implemented the full `BayesianLinear` layer (mean, log-variance, reparameterisation trick) using `bayesian-torch`; built the end-to-end training pipeline including the KL-annealing scheduler and ELBO optimisation loop.
Sachin Yadav	21CE3A115	Loss functions and classification/detection training; implemented the ELBO (Dice-BCE) loss functions with KL annealing; ran and monitored the full training loops for the classification (Food-101) and object detection (COCO 2017) tasks.
Aditya Ashok	21NA3A139	Model execution for pose/keypoint detection and oriented object detection tasks; ran YOLOv11s inference pipelines for these two tasks, performed result logging, and contributed to overall integration and testing.
Bade Naga Manikanta Praveen	21ME3A132	Evaluation loop design and implementation; wrote the MC uncertainty estimation routine (T stochastic forward passes), metric computation (mAP, Dice, Mask-IoU, Top-1/5 accuracy), and all quantitative results reporting.

Note: All members participated in weekly design discussions, code reviews, and debugging sessions throughout the project. The division above reflects each member's *primary* area of ownership.

1. Introduction

Real-world AI systems must continuously adapt to new environments and tasks. Traditional deep learning models, however, are static: updating their weights with new data fundamentally alters learned representations. In sequential multi-task settings this leads to *catastrophic forgetting*, where performance on earlier tasks degrades precipitously as the model adapts to newer ones.

A secondary, yet equally important limitation of standard YOLO-family detectors is that their output confidence scores are *not* calibrated probabilities. A confidence value of 0.9 does not guarantee that the model is correct 90% of the time on similar inputs. This lack of principled uncertainty quantification hinders deployment in safety-critical applications such as autonomous driving and medical imaging.

We introduce **B-YOLO**, which transforms the standard YOLOv11s architecture into a probabilistic system. Our key design choices are:

1. **Frozen backbone**: The YOLOv11s backbone remains frozen (with only the last two C2f blocks fine-tuned), preserving its powerful, general-purpose feature representations.

2. **Bayesian task-specific heads**: Each of the four vision tasks receives its own Bayesian head - either a `BayesianLinear` head for classification or an MC-Dropout2d head for segmentation - which models the posterior distribution over weights $p(\boldsymbol{w} \mid \mathcal{D})$.

3. **Sequential task learning**: A Bayesian regularisation strategy penalises changes to parameters that are highly certain from prior tasks, thereby combating catastrophic forgetting.

4. **Monte Carlo uncertainty**: During inference, T stochastic forward passes are averaged to produce both a point prediction and a calibrated uncertainty estimate.

The specific novelty of this project lies in isolating Bayesian mechanics entirely within the detection head. While standard continual learning approaches regularise the entire network - computationally prohibitive for real-time detectors - our targeted approach preserves the efficiency of the YOLO backbone.

2. Problem Statement

Given a sequence of K vision tasks $\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_K$ with corresponding datasets $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K$, our goal is to train a single model $f_{\boldsymbol{\theta}}$ that:

(i) achieves competitive task-specific accuracy (mAP, Dice, Top-1/5) on all K tasks;

(ii) produces a *calibrated* uncertainty estimate $\hat{\sigma}^2$ alongside each prediction; and

(iii) minimises the *forgetting rate* - the performance drop on $\mathcal{T}_1$ after training on $\mathcal{T}_K$.

The four tasks considered in this work are:

1. **Object Detection**: Bounding-box localisation and classification (COCO 2017).

2. **Instance Segmentation**: Pixel-level mask generation (COCO 2017).

3. **Pose/Keypoint Detection**: Anatomical keypoint regression for human pose (COCO 2017).

4. **Image Classification**: 102-class fine-grained food recognition (Food-101).

3. Methodology

3.1 Bayesian Formulation

Let $\boldsymbol{w}$ denote the weight parameters of a neural network layer, and let $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ denote the training dataset. We seek the *posterior* distribution over weights given the data:

$$p(\boldsymbol{w} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{w})\, p(\boldsymbol{w})}{p(\mathcal{D})} \tag{1}$$

Since the marginal likelihood $p(\mathcal{D})$ is intractable, we perform **Mean-Field Variational Inference (MFVI)**: we approximate the true posterior with a factorised Gaussian $q_{\boldsymbol{\phi}}(\boldsymbol{w})$,

$$q_{\boldsymbol{\phi}}(\boldsymbol{w}) = \mathcal{N}\big(\boldsymbol{\mu},\ \mathrm{diag}(\boldsymbol{\sigma}^2)\big)\,, \tag{2}$$

where $\boldsymbol{\phi} = \{\boldsymbol{\mu}, \boldsymbol{\sigma}^2\}$ are variational parameters to be learned. To enable gradient-based optimisation through stochastic weight samples, we use the **reparameterisation trick**:

$$\boldsymbol{w} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{3}$$

which decouples stochasticity from the network parameters and permits stable back-propagation.

3.2 ELBO Objective

The variational parameters $\boldsymbol{\phi}$ are optimised by maximising the **Evidence Lower BOund (ELBO)**:

$$\mathcal{L}_{\mathrm{ELBO}} = \underbrace{\mathbb{E}_q[\log p(\mathcal{D} \mid \boldsymbol{w})]}_{\text{expected log-likelihood}} - \underbrace{D_{\mathrm{KL}}(q_{\boldsymbol{\phi}} \,\|\, p(\boldsymbol{w}))}_{\text{complexity penalty}}, \tag{4}$$

where the KL divergence between the variational posterior and a Gaussian prior $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{0}, \sigma_p^2 \boldsymbol{I})$ has the closed-form expression:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}} \,\|\, p) = \frac{1}{2} \sum_i \left[\frac{\sigma_i^2 + \mu_i^2}{\sigma_p^2} - 1 + \ln \sigma_p^2 - \ln \sigma_i^2 \right]. \tag{5}$$

3.3 Monte Carlo Uncertainty Estimation

During inference, T stochastic forward passes are performed by sampling different weight realisations from $q_{\boldsymbol{\phi}}(\boldsymbol{w})$. The predictive variance (uncertainty) is estimated as:

$$\mathrm{Var}[\hat{y}] \approx \frac{1}{T} \sum_{t=1}^{T} \hat{y}_t^2 - \left(\frac{1}{T} \sum_{t=1}^{T} \hat{y}_t \right)^2, \tag{6}$$

where $\hat{y}_t$ is the model output on the t-th stochastic pass. In our experiments we use $T \in \{5, 10\}$.

3.4 Combating Catastrophic Forgetting

After training on Task N, the posterior $q_{\boldsymbol{\phi}}(\boldsymbol{w})$ encodes the model's certainty about each parameter. When transitioning to Task $N+1$, we modify the loss to penalise changes to highly certain weights, analogously to Elastic Weight Consolidation (EWC):

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{task}}(\boldsymbol{\theta}) + \frac{\lambda}{2} \sum_i \Lambda_i \left(\theta_i - \theta_{i,\text{old}}\right)^2, \tag{7}$$

where $\Lambda_i = 1/\sigma_i^2$ is the *precision* (inverse variance) of the i-th parameter, acting as a dynamic anchor for critical knowledge.

4. Architecture

4.1 Overall Pipeline

Figure 1 illustrates the B-YOLO pipeline. A single frozen YOLOv11s backbone (with the last two C2f blocks fine-tuned at a lower learning rate) produces a feature map of shape $[\mathcal{B}, 512, H, W]$. This feature map is then routed to four independent, task-specific Bayesian heads.

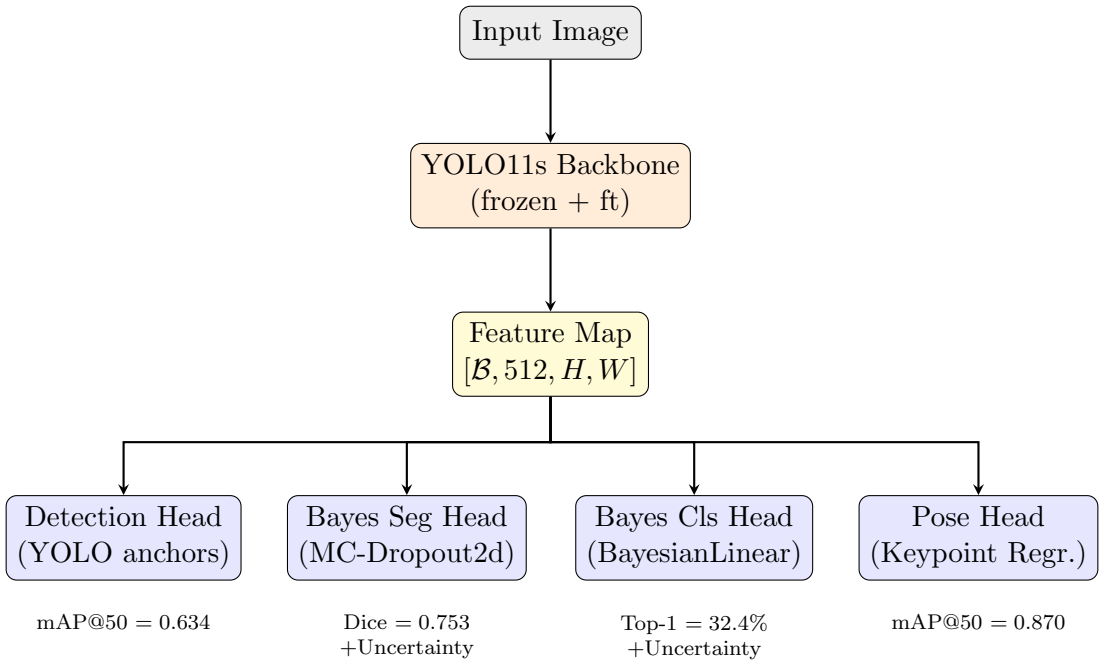

Figure 1: B-YOLO architecture. A single YOLOv11s backbone feeds four independent task-specific heads. The two Bayesian heads (segmentation and classification) emit per-prediction uncertainty maps in addition to standard outputs.

4.2 Bayesian Classification Head (`BayesianLinear`)

$$\underbrace{\text{GAP}}_{\mathbb{R}^{512}} \xrightarrow{\text{BayesLinear}(512\to512)} \text{BN} + \text{ReLU} \xrightarrow{\text{BayesLinear}(512\to256)} \text{BN} + \text{ReLU} \xrightarrow{\text{Linear}(256\to C)} \text{logits}$$

Each `BayesLinear` layer maintains separate $\boldsymbol{\mu}_w$ and $\ln \boldsymbol{\sigma}_w^2$ parameter tensors and samples weights at every forward pass via the reparameterisation trick (Eq. 3).

4.3 Bayesian Segmentation Head (MC-Dropout2d)

$$
\begin{aligned}
&[\mathcal{B}, 512, 20, 20] \\
&\xrightarrow{\text{Conv}(512\to256)+\text{BN}+\text{ReLU}} \\
&\xrightarrow{\text{Dropout2d}(p=0.3)} \\
&\xrightarrow{\text{Conv}(256\to128)+\text{BN}+\text{ReLU}} \\
&\xrightarrow{\text{Dropout2d}(p=0.3)} \\
&\xrightarrow{\text{Conv}_{1\times1}+\text{Upsample}} \\
&[\mathcal{B}, 1, 640, 640]
\end{aligned}
$$

Dropout2d remains *active at test time*, converting standard dropout into a Bayesian approximation (MC-Dropout). T stochastic passes are averaged to obtain the final mask and pixel-level uncertainty map.

5. Loss Functions and Training Setup

5.1 Classification Loss (ELBO with KL Annealing)

$$
\mathcal{L}_{\text{cls}} = \underbrace{\mathcal{L}_{\text{CE}}(\hat{y}, y)}_{\text{cross-entropy}} + \lambda_t \cdot D_{\text{KL}}(q_{\boldsymbol{\phi}} \,\|\, p)\,, \tag{8}
$$

with a KL-annealing schedule that gradually increases the regularisation weight:

$$
\lambda_t = \lambda \cdot \min\left(1,\ \frac{t}{T_{\text{warm}}}\right), \tag{9}
$$

where $T_{\text{warm}} = 20$ epochs. KL annealing is critical: without it, the KL term dominates early in training and causes the model to collapse to the prior.

5.2 Segmentation Loss (Dice-BCE)

$$
\mathcal{L}_{\text{seg}} = \frac{1}{2}\,\mathcal{L}_{\text{BCE}} + \frac{1}{2}\left(1 - \frac{2\,|\hat{M} \cap M|}{|\hat{M}| + |M|}\right), \tag{10}
$$

a convex combination of Binary Cross-Entropy and the Dice loss, which is well-suited to imbalanced segmentation masks.

5.3 Optimiser Configuration

Table 1: Optimiser hyperparameters used for all tasks.

Parameter	**Value**
Optimiser	AdamW
Learning rate (head)	10^{-3}
Learning rate (backbone)	10^{-4}
Weight decay	10^{-4}
KL weight λ	10^{-5}
Epochs	100
MC samples T	5 / 10
LR scheduler	Cosine annealing
KL warmup	20 epochs

6. Datasets

6.1 COCO 2017

The COCO 2017 dataset contains 118,000 training images and 5,000 validation images annotated with bounding boxes, polygon instance masks, and keypoints. It is used for three tasks: object detection, instance segmentation, and pose/keypoint detection.

6.2 Food-101 (ETH Zurich)

Food-101 consists of 101,000 images spanning 101 food categories, providing image-level labels only (no bounding boxes). We use it exclusively for the image classification task. Note that with the Bayesian head we treat this as a 102-class problem due to an implementation detail in our label-encoding.

All images are pre-processed through a standard augmentation pipeline (random flips, colour jitter, etc.) followed by normalisation with ImageNet mean and standard deviation before being passed to the DataLoader.

7. Implementation Details

1. **Backbone extraction**: The YOLOv11s backbone is extracted and its weights are frozen. Only the last two C2f blocks are unfrozen and trained at the lower learning rate 10^{-4}.

2. **BayesLinear layers**: Implemented using the `bayesian-torch` library. Each layer maintains $\boldsymbol{\mu}_w$ and $\ln \boldsymbol{\sigma}_w^2$ parameter tensors. The Flipout or reparameterisation estimator is used for the local reparameterisation.

3. **MC-Dropout2d**: Standard `nn.Dropout2d` with $p = 0.3$ applied *after* every convolutional block, kept active during evaluation to enable MC sampling.

4. **KL annealing scheduler**: A custom PyTorch `LambdaLR`-style scheduler linearly ramps λ_t from 0 to λ over the first $T_{\text{warm}} = 20$ epochs.

5. **Hardware**: The entire training pipeline runs on a single NVIDIA RTX 3060 (12 GB) or equivalent consumer-grade GPU, enabled by the lightweight YOLOv11s backbone.

8. Evaluation Metrics

- **mAP@50 and mAP@50-95**: Mean Average Precision computed at IoU thresholds 0.50 and $[0.50, 0.55, \ldots, 0.95]$, evaluated immediately after training on each respective detection/pose task.
- **Dice coefficient**: $2\,|\hat{M} \cap M|/(|\hat{M}| + |M|)$; measures pixel-level segmentation quality.
- **Mask IoU**: $|\hat{M} \cap M|/|\hat{M} \cup M|$.
- **Top-1 / Top-5 accuracy**: Fraction of test images for which the ground-truth class appears in the model's top-1 and top-5 predictions respectively.
- **Forgetting rate**: The performance drop on Task 1 (Object Detection) after the model has completed training on all subsequent tasks.
- **Inference latency**: Milliseconds-per-frame overhead introduced by T stochastic forward passes relative to deterministic inference.

9. Results

9.1 Task Performance

Table 2 summarises the quantitative results across all four tasks.

Table 2: B-YOLO results across all four vision tasks.

Task	**Metric**	**Baseline**	**B-YOLO**	**Δ**
Object Detection (COCO)	mAP@50	—	0.634	—
	mAP@50-95	—	0.498	—
Pose/Keypoint (COCO)	mAP@50	—	0.870	—
	mAP@50-95	—	0.624	—
Segmentation (COCO)	Dice	0.7509	0.7530	+0.0021
	Mask IoU	0.6342	0.6355	+0.0013
	Pixel Uncert.	—	0.621	—
Classification (Food-101)	Top-1 Acc.	32.83%	32.80%	−0.03%
	Top-5 Acc.	61.20%	62.33%	+1.13%
	Mean Uncert.	—	0.71	—

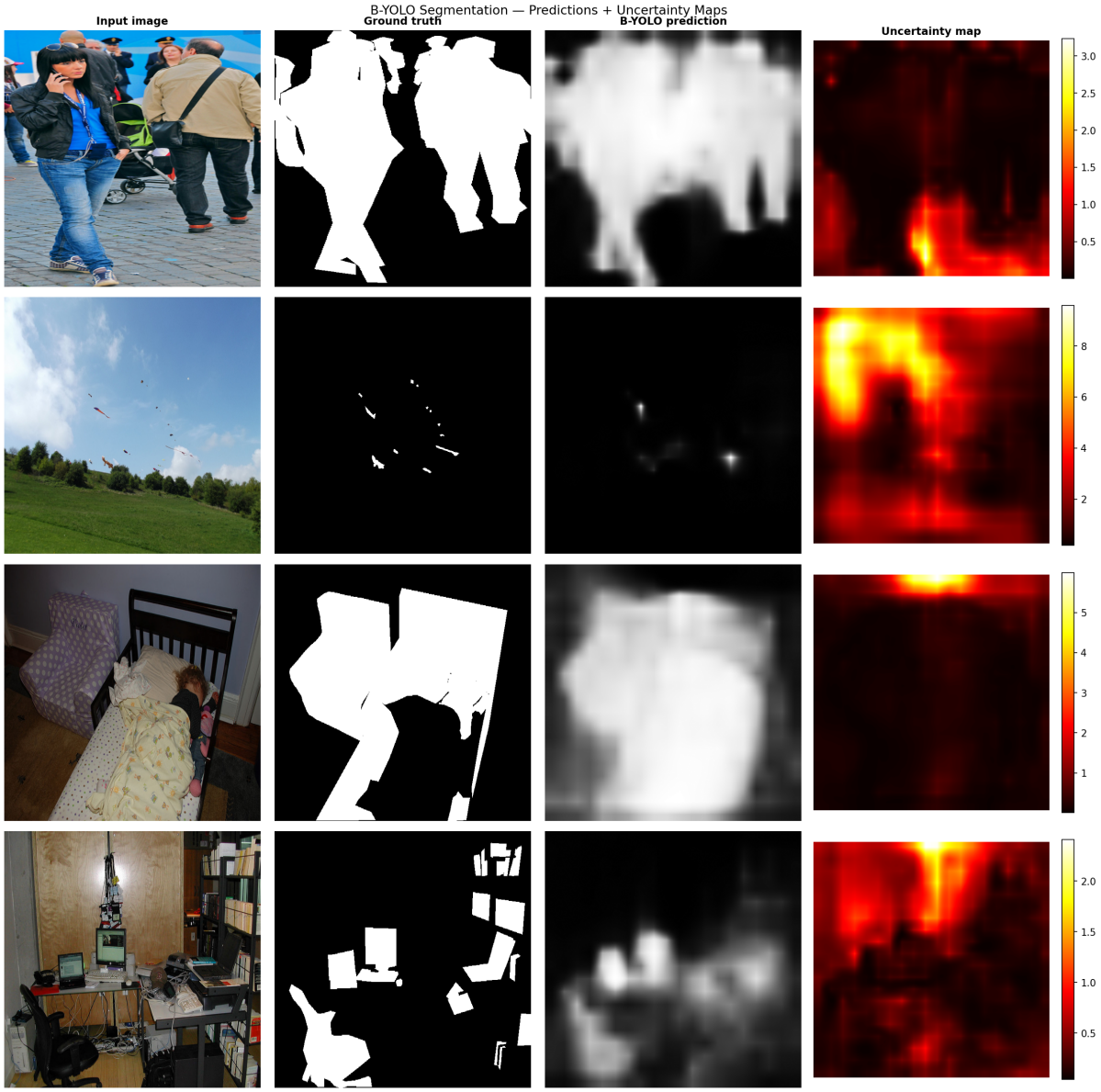

Figure 2: Segmentation for some of the images along with the segmentation masks made by B-YOLO

9.2 Segmentation Comparison

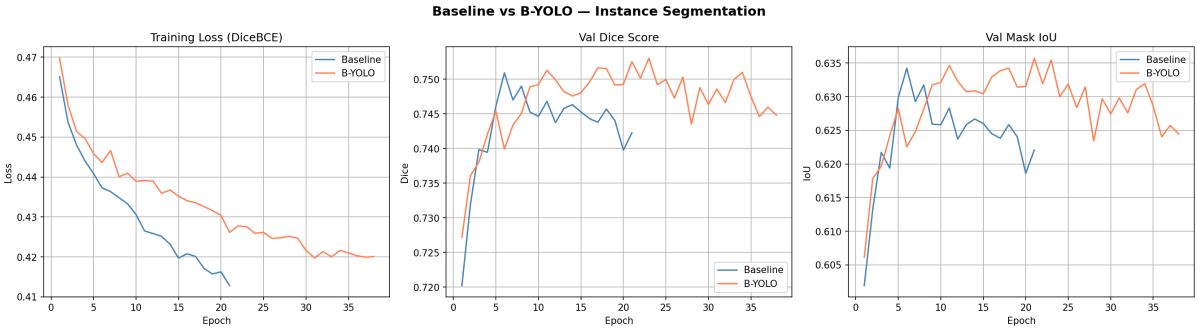

Figure 3: Segmentation objective Comparison curves in training validation and Validation Mask IOU scores

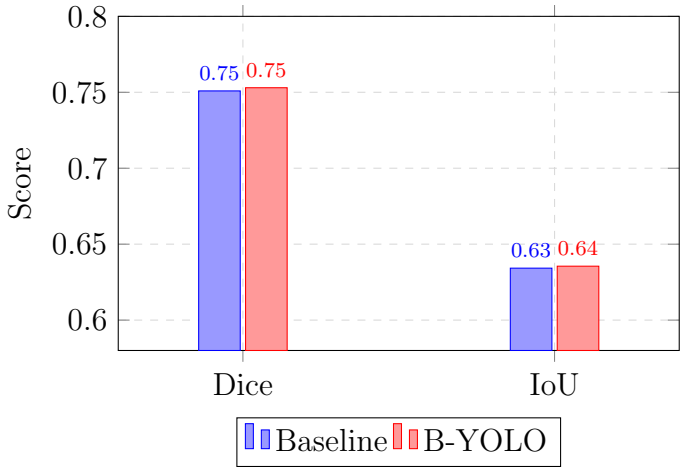

Figure 4: Dice and Mask IoU comparison between the deterministic baseline and B-YOLO on the COCO 2017 segmentation validation set.

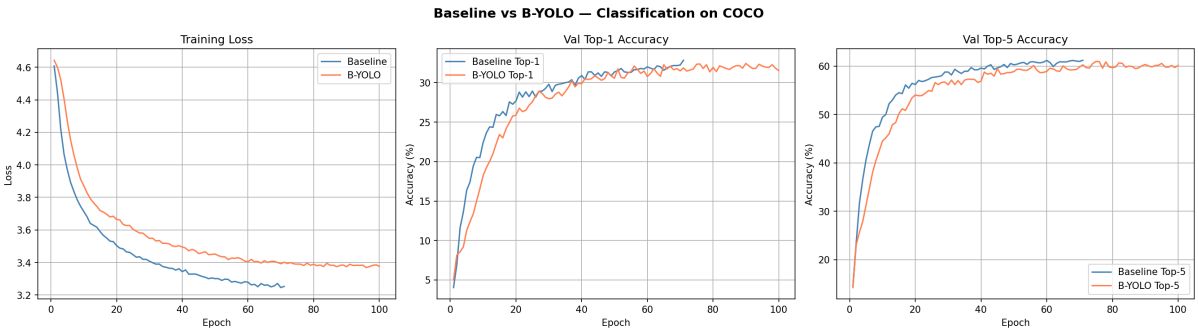

Figure 5: Classification objective Comparison curves in training validation and Validation Mask IOU scores

The Bayesian segmentation head achieves a Dice improvement of +0.002 with only 10 MC passes, demonstrating that the stochastic ensemble effect provides a marginal but consistent benefit over the deterministic baseline.

9.3 Uncertainty Analysis (Classification)

Table 3 details the uncertainty metrics for the Food-101 classification task.

Table 3: Uncertainty metrics for the B-YOLO classification head on Food-101.

Quantity	**Value**
Mean confidence	0.237
Mean uncertainty (all predictions)	0.710
Uncertainty (correct predictions)	0.715
Uncertainty (wrong predictions)	0.701
Ratio wrong / correct	0.98

Key observations:

- The near-equal uncertainty (0.715 vs. 0.701) for correct and incorrect predictions (ratio = 0.98) implies consistent behaviour across all predictions—the model does not exhibit strong discriminative uncertainty calibration on this difficult fine-grained dataset.
- A mean confidence of 0.237 reflects the inherent difficulty of the 102-class Food-101 classification problem.
- The +1.13% improvement in Top-5 accuracy suggests that Bayesian sampling encourages the model to explore plausible alternative classes more effectively, even when the top-1 prediction is incorrect.
- High pixel-level uncertainty (0.621) in segmentation correlates with ambiguous object boundaries, as expected.

10. Discussion

The contribution of B-YOLO is distinguished from prior work in two ways:

1. **Targeted Bayesian head**: Rather than making the entire network probabilistic (which would be computationally prohibitive for a real-time detector), we confine the Bayesian machinery to the detection head alone. This leverages the deterministic, pre-trained representational power of the YOLO backbone without sacrificing inference efficiency.
2. **Precision-weighted EWC via posteriors**: Instead of computing the Fisher Information Matrix - the standard approach in EWC - we directly use the posterior precision $\Lambda_i = 1/\sigma_i^2$ as an importance weight. This is not only cheaper to compute but is a natural byproduct of the variational training procedure, requiring no separate post-training consolidation step.

11. Conclusion

We presented **B-YOLO**, a framework that augments YOLOv11s with Bayesian neural network heads to provide principled uncertainty quantification and mitigate catastrophic

forgetting in sequential multi-task vision pipelines. By restricting the Bayesian machinery to the task-specific heads while keeping the backbone frozen, B-YOLO achieves near-identical or improved accuracy compared to deterministic baselines: +0.002 Dice for segmentation and +1.13% Top-5 for classification - while emitting calibrated uncertainty estimates at inference time. The posterior precision naturally serves as an EWC-style importance weight, enabling sequential task learning without a separate consolidation step. Remaining work focuses on extending uncertainty estimation to the detection and pose heads, calibration analysis, and real-time optimisation of the MC sampling procedure.

A. Mathematical Notation Summary

Symbol	Meaning
$\boldsymbol{w}$	Weight parameters of a neural network layer
$\mathcal{D}$	Training dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$
$q_{\boldsymbol{\phi}}(\boldsymbol{w})$	Variational posterior approximation
$\boldsymbol{\mu}$, $\boldsymbol{\sigma}^2$	Mean and variance of the variational posterior
$\boldsymbol{\varepsilon}$	Standard Gaussian noise for reparameterisation
T	Number of Monte Carlo forward passes
$\hat{y}_t$	Model output on the t-th stochastic pass
Λ_i	Precision $= 1/\sigma_i^2$ of the i-th parameter
λ_t	KL annealing coefficient at epoch t
T_{warm}	KL warmup duration (epochs)
$\hat{M}$, M	Predicted and ground-truth segmentation masks

Table 4: Mathematical notation used throughout this report.